Conectar Gemini, Claude y GitHub

https://gemini.google.com/app/946f96898609eba9

User prompt: te cuento, quiero hacer que puedas comunicarte con el claude para que te pueda pedir videos de resumen y targetas, tambien quiero conectar el claude y el gamini con el github, soy un novato, tu ves viable poder hacer esto siendo novato, ves poniendome paso a paso muy sencillo como poder hacerlo

Response: ¡Es totalmente viable! De hecho, este tipo de conexiones es el siguiente paso natural después de haber montado proyectos web como tu herramienta AgentIT en Vercel. Como ya sabes manejar repositorios y hacer despliegues, tienes la mitad del terreno técnico ganado.

Para que dos inteligencias artificiales "hablen" entre sí y guarden cosas en GitHub, no lo hacen por arte de magia; necesitan un intermediario que recoja el mensaje de una y se lo entregue a la otra.

Aquí tienes el plan de ataque paso a paso, diseñado para que lo puedas ir montando sin complicarte la vida.

Paso 1: Entender las piezas del puzzle (Las APIs)

Para que tus programas se comuniquen con Gemini, Claude y GitHub, necesitas usar sus **APIs**. Una API es como la ventanilla de un restaurante: tú (el puente) le pasas un papel con tu pedido (el prompt), y el cocinero (Gemini/Claude) te devuelve el plato preparado (el texto o resumen).

Para que te dejen hacer pedidos, necesitas una "tarjeta de identificación" para cada uno, llamada **API Key**.

- **Gemini:** Se consigue gratis en *Google AI Studio*.
- **Claude:** Se consigue en la *Anthropic Console*.
- **GitHub:** Se llama *Personal Access Token* y se saca desde los ajustes de tu perfil (Developer settings).

Paso 2: Elegir el "Puente" (El intermediario)

Tienes dos formas de hacer que estas piezas hablen entre sí. Como estás empezando, te recomiendo elegir la que te resulte más cómoda:

1. **La vía visual (Sin código):** Usar una plataforma como **Make.com** o **Zapier**. Es como jugar con piezas de Lego. Arrastras un bloque de Gemini, lo unes con una flecha a un bloque de Claude, y de ahí a un bloque de GitHub. Es lo más rápido para novatos.
2. **La vía del código (Python o Node.js):** Si quieres practicar programación y aprovechar lo que estás viendo en la universidad, puedes crear un pequeño script. Este script le mandará la información a las APIs usando código y lo puedes alojar gratis donde ya conoces, en Vercel o directamente correrlo en tu ordenador.

Paso 3: El flujo de trabajo

Si decides montar esto (por ejemplo, en Make.com o en tu propio código), el camino lógico que tendrás que construir será este:

1. **El Detonante (Trigger):** Tú envías un enlace de un vídeo o un texto a tu "puente".
2. **Turno de Gemini:** El puente le envía la información a Gemini y le pide: *"Sácame los puntos clave de este tema"*. Gemini devuelve el texto.
3. **Turno de Claude:** El puente coge lo que dijo Gemini y se lo manda a Claude con la instrucción: *"Crea tarjetas de estudio (flashcards) y un resumen estructurado a partir de este texto"*.
4. **Guardado en GitHub:** El puente coge el resultado final de Claude y usa la API de GitHub para crear automáticamente un archivo `.md` (Markdown) dentro de tu repositorio, guardando tu resumen para siempre.

Para empezar y no saturarte, el mejor primer paso es simplemente **conseguir las API Keys**. Una vez las tengas guardadas en un bloc de notas, conectar los cables es mucho más fácil.

User prompt: explicame paso a paso, una guia para poder saber como hacer los de las apis (conectar las IA's i que hablen) y tambien sobre todo el funcionamiento de todo y tambien como funciona el github la carga de archivos y como decirles a las ias que utilicen el contenido y la carpeta con los archivos

Response: El funcionamiento de todo este sistema se basa en una "carrera de relevos". Las APIs de Gemini, Claude y GitHub no se hablan entre ellas por arte de magia ni se conectan directamente. Necesitan un **director de orquesta** (un pequeño programa hecho por ti, por ejemplo en Python o JavaScript) que se encargue de coger la información de un sitio y llevarla al otro.

1. ¿Cómo funciona la subida y lectura de archivos en GitHub?

Como ya tienes experiencia con repositorios, sabes que normalmente haces un `git commit` y `git push`. Con la API de GitHub, el proceso cambia:

- **Para leer archivos (Que la IA los use):** Tu programa hace una petición a la API de GitHub pidiendo el contenido de un archivo concreto. GitHub te devuelve el texto.
- **Para subir archivos (Guardar el resultado):** La API de GitHub no permite "arrastrar y soltar". Tienes que coger el texto que ha generado la IA, convertirlo a un formato de texto seguro llamado **Base64** (es un estándar de codificación), y enviarle a GitHub un mensaje diciendo: *"Crea un archivo llamado `resumen.md` en esta carpeta, y aquí tienes el contenido en Base64"*. GitHub lo decodifica y lo guarda en tu repositorio.

2. ¿Cómo le dices a las IAs que utilicen esos archivos?

Las IAs no pueden "entrar" a tu ordenador ni a tu GitHub a leer una carpeta por sí solas. Tienes que **inyectar** el contenido en el mensaje que les envías. El truco está en separar las instrucciones del contenido usando lo que se llama un *System Prompt* (el rol) y un *User Prompt* (la orden y los datos):

> **Ejemplo de lo que tu programa le envía a la IA:** "Eres un experto en crear tarjetas de estudio. *(Instrucción)* Basándote estrictamente en el siguiente texto, genera 5 tarjetas. *(Orden)*

TEXTO A UTILIZAR: [Aquí tu programa pega automáticamente todo el texto que sacó de GitHub o del vídeo]"

3. La Guía Paso a Paso (El Flujo del Programa)

Aquí tienes la secuencia exacta de lo que tiene que hacer el código de tu "puente" o "director de orquesta" cada vez que lo ejecutes.

1 **Extraer el contenido original**

Preparación

Tu programa recibe el enlace de un vídeo o lee un archivo de tu carpeta de GitHub mediante su API. Guarda todo ese texto crudo en una variable (la memoria temporal del programa). *Verification:* El programa imprime en pantalla el texto extraído para confirmar que no está vacío.

2 **Llamada a la API de Gemini**

Primer procesamiento

El programa envía una petición web a los servidores de Google. En el "paquete" de envío, mete tu API Key de Gemini y el mensaje: *"Resume los conceptos clave de este texto: [TEXTO CRUDO]"*. El programa espera unos segundos hasta que Gemini responde y guarda el resumen en una nueva variable. *Verification:* El programa recibe un texto estructurado con el resumen y lo muestra en consola.

3 **Llamada a la API de Claude**

Segundo procesamiento

Ahora, el programa coge esa variable con el resumen de Gemini y hace una nueva petición a los servidores de Anthropic (Claude). Le envía tu API Key de Claude y el mensaje: *"A partir de este resumen, créame tarjetas de memoria (Flashcards) en formato Markdown: [RESUMEN DE GEMINI]"*. *Verification:* Claude devuelve un texto con formato de tarjetas (por ejemplo, Pregunta/Respuesta).

4 **Guardar el resultado en GitHub**

Almacenamiento final

El programa coge el resultado final de Claude, lo codifica en Base64, y hace una última petición a la API de GitHub (usando tu Personal Access Token). Le indica la ruta del repositorio (ej. `AgentIT/apuntes/tarjetas.md`) y el contenido. *Verification:* Entras a tu cuenta de GitHub en el navegador y ves el nuevo archivo `.md` creado correctamente.

Para llevar esto a la práctica, lo ideal es escribir un pequeño script en Python, ya que librerías como `requests` hacen que enviar estos "paquetes" de información a las APIs sea cuestión de 4 o 5 líneas de código por cada paso.

User prompt: una pregunta antes de emnpezar todo esto es gratis?

Response: rrSí, ¡puedes montar todo este sistema completamente gratis! Las tres herramientas (Gemini, Claude y GitHub) ofrecen acceso gratuito para desarrolladores que están empezando o haciendo proyectos personales.

Aquí te detallo qué te ofrece cada una sin pagar ni un euro:

1. GitHub API (Siempre Gratis)

GitHub es la más generosa de las tres. Al usar un Personal Access Token (PAT) con tu cuenta gratuita, tienes un límite de **5.000 peticiones por hora**.

- **¿Es suficiente?** Para tu proyecto, vas a gastar quizás 2 o 3 peticiones por cada resumen que guardes. Tendrías que estar generando miles de resúmenes cada hora para llegar al límite, así que por aquí no vas a tener ningún problema.

2. Google Gemini API (Nivel Gratuito de Google AI Studio)

Google te permite usar su modelo de inteligencia artificial de forma gratuita si usas "Google AI Studio".

- **Límites:** El plan gratuito te da **15 peticiones por minuto** o hasta **1.000 peticiones al día**.
- **¿Es suficiente?** De sobra. Como tú enviarás los textos uno a uno, jamás pasarás de las 15 peticiones por minuto.
- **La única "pega":** Al usar el nivel gratuito de Gemini, Google se reserva el derecho de usar los datos que le envíes para mejorar sus modelos de IA (algo que no pasa si pagas). Como son apuntes de clase y no secretos de estado, no debería importarte.

3. Anthropic Claude API (Crédito de Prueba)

Claude es la más restrictiva. Anthropic no tiene un "nivel gratuito para siempre" como Google para usar su API.

- **Lo que te dan:** Cuando te registras por primera vez en la Anthropic Console y verificas tu número de teléfono, te regalan **5$ en créditos de prueba**.
- **¿Es suficiente?** La API se cobra por "tokens" (cantidad de texto que lees y generas). Usando el modelo *Claude 3.5 Haiku* (que es rápido y barato, ideal para hacer tarjetas), esos 5 dólares te dan para procesar casi 1 millón de palabras. Tendrás de sobra para montar el proyecto, probarlo durante semanas y sacar tus apuntes de la universidad. Cuando se gasten esos 5 dólares, tendrías que recargar saldo (por ejemplo, con 5 euros tendrías para meses de uso esporádico).

En Resumen

- **GitHub:** Gratis.
- **Gemini:** Gratis para siempre (con límite diario que no alcanzarás).
- **Claude:** Te dan saldo de regalo para empezar (5$), que te durará semanas o meses de pruebas.

Para empezar a programar el "puente" y sacar las API Keys, **no te van a pedir tarjeta de crédito en ninguna de las tres plataformas**. ¿Te parece bien si empezamos sacando la de Google Gemini, que es la más fácil?

Sources

1. Google AI Studio Pricing (2026): Free Tier, Plans & API Costs
2. GitHub API Rate Limit Guide: Best Practices for AI Agents - Fastio
3. Gemini API Pricing and Quotas: Complete 2026 Guide with Cost
4. How to Get a Free Anthropic API Key in 2026 (Plus Free Credits)
5. Claude API Free Credits: Every Way to Get Them (2026)
6. Gemini Developer API pricing - Google AI Studio

User prompt: y en si el vercel para que podria utilizarlo?

Response: Como ya usaste Vercel para desplegar tu proyecto AgentIT, ya sabes que es una plataforma genial para alojar páginas web. Pero para este proyecto de las IAs, Vercel te serviría para algo muy específico: **ser el "ordenador" que ejecuta tu programa las 24 horas del día.**

Recuerda que antes hablamos de que necesitas construir un "puente" (tu código) que conecte a Gemini, Claude y GitHub. Tienes dos opciones para hacer funcionar ese puente:

1. En tu propio ordenador (Ideal para empezar y hacer pruebas): Escribes el código en tu portátil Asus, le das a "Ejecutar" y el programa hace su magia.

- *El problema:* Solo funciona si tú estás delante del ordenador, lo tienes encendido y ejecutas el código manualmente.

2. En Vercel (Para automatizarlo del todo): Vercel no solo sirve para alojar el "diseño" de una web (HTML/CSS), también tiene algo llamado **Serverless Functions** (funciones en la nube). Puedes subir ese pequeño script de código a Vercel.

- *La ventaja:* Al subirlo, Vercel te dará un enlace web (una URL) secreto. Tu código estará "durmiendo" en sus servidores. Cuando tú entres a esa URL desde tu móvil, desde la universidad o desde cualquier sitio, Vercel despertará tu código, ejecutará la conexión entre Gemini, Claude y GitHub, y volverá a apagarse.

¿Cómo quedaría el proyecto completo usando Vercel?

Si unes lo que ya sabes de AgentIT con este nuevo proyecto, podrías crear una página web súper sencilla alojada en Vercel con un solo campo de texto.

Tú pegas ahí los apuntes de la universidad o un texto sobre electromagnetismo, le das a un botón que diga "Generar Tarjetas", y Vercel se encarga por detrás de hablar con Gemini, luego con Claude, y finalmente guardar el `.md` en tu GitHub sin que tengas que abrir ningún programa de código en tu portátil.

Para ir paso a paso y no agobiarte, lo mejor es hacer que el código funcione primero en tu propio ordenador. Una vez veas que los archivos aparecen en tu GitHub por arte de magia, subir ese código a Vercel es el último paso.

User prompt: tambien lo de las apis y el conector este de pyhton que me has dicho especifica mas

Response: Para que lo veas con mentalidad de ingeniero (ya que estás dándole al C y a MATLAB en la carrera), el "conector" de Python no es más que un pequeño programa que hace de intermediario. Como en C, tendrás variables y funciones, pero Python es muchísimo más amigable para conectarse a internet.

Ese conector va a ser un simple archivo de texto llamado, por ejemplo, `puente.py`, que guardarás en tu portátil Asus.

Aquí te despiezo exactamente cómo funciona por dentro.

1. La herramienta estrella: La librería `requests`

Para que tu programa en Python pueda enviar mensajes por internet a los servidores de Google, Anthropic o GitHub, usa una herramienta prefabricada llamada **`requests`** (peticiones). Piensa en `requests` como el cartero. Tú le das una dirección web (URL), una carta (tu texto) y un sello (tu API Key), y él se encarga de ir y volver con la respuesta.

2. La anatomía de una petición (Cómo se habla con una API)

Cada vez que tu código de Python hable con una de las tres IAs, siempre va a construir un "paquete" con tres partes fundamentales:

- **El Endpoint (La dirección):** Es la URL secreta de la API. Por ejemplo, para Gemini es algo como `[https://generativelanguage.googleapis.com/](https://generativelanguage.googleapis.com/)`....
- **Los Headers (La cabecera):** Aquí es donde metes tu "carnet de identidad" para que te dejen pasar. Es decir, tu **API Key**.
- **El Payload (El cuerpo):** Aquí va el contenido. Se envía en un formato universal llamado **JSON** (que es como un diccionario de datos). Aquí es donde pones tu instrucción: *"Resume este texto"*.

3. El esqueleto de tu código en Python

Tu archivo `puente.py` tendrá una estructura muy lógica y ordenada, dividida en 3 funciones principales. Así es como pensará tu programa:

A. Función 1: `pedir_resumen_a_gemini(texto_original)`

- El código coge el texto de tus apuntes y usa `requests.post()` para enviarlo a la URL de Gemini.
- Gemini procesa y devuelve un archivo JSON lleno de datos técnicos.
- Python busca dentro de ese JSON solo la parte que contiene el texto del resumen y la guarda en una variable.

B. Función 2: `crear_tarjetas_con_claude(resumen_gemini)`

- El código coge esa variable anterior.
- Usa `requests.post()` para enviarla a la URL de Claude con la instrucción: *"Crea tarjetas con este resumen"*.
- Claude responde con otro JSON. Python extrae las tarjetas generadas y las guarda.

C. Función 3: `subir_a_github(tarjetas_claude)`

- Aquí Python hace un truco. La API de GitHub exige que el texto esté codificado en un formato llamado Base64. Python usa una librería interna (`import base64`) para traducir el

texto de Claude.

- Luego usa `requests.put()` (poner) apuntando a la URL de tu repositorio (ej: `[https://api.github.com/repos/tuUsuario/tuRepo/contents/apuntes.md](https://api.github.com/repos/tuUsuario/tuRepo/contents/apuntes.md)`).
- Le pasa el texto en Base64 y el *Personal Access Token* de GitHub. ¡Bum! Archivo creado en tu repositorio.

¿Cómo se ve esto en código real?

Para que veas que no impone tanto, así es como se ve *realmente* una llamada a una API en Python (es casi como leer en inglés):

```
import requests

# 1. Preparas los datos
url_de_la_api = "https://api.sitio.com/generar"
mi_api_key = "AQUI_PONES_TU_CLAVE_SECRETA"
mi_mensaje = {"texto": "Resume la teoría de circuitos"}

# 2. Envías al cartero (requests)
respuesta = requests.post(
    url_de_la_api,
    headers={"Authorization": mi_api_key},
    json=mi_mensaje
)

# 3. Lees lo que te han devuelto
print(respuesta.json())
```

Como ves, la lógica de programación es sencilla: preparas el paquete, lo envías y lees la respuesta. Al poner las tres funciones una detrás de otra, creas la cadena completa.
